Отчет по выполнению практикума

Дисциплина
Интеграция и управление приложениями
на удаленном сервере

Текущий рейтинг Kittygram

Вторая работа
Реализация и проверка REST API
в Docker Desktop

ФИО студента: Солопин Егор Алексеевич

Группа: ПИ2у/23б

Преподаватель: Данилов А. В.

Дата выполнения: 26 октября 2026 года

Исходный репозиторий проекта

https://github.com/yandex-praktikum/kittygram

Реализованная версия передана в архиве материалов вместе с исходниками, инструкцией запуска и проверочными запросами.

2026

1 Цель и итоговый результат

Цель работы — реализовать и проверить базовый REST API проекта Kittygram: модели котов и достижений, CRUD, аутентификацию, разграничение доступа, поиск, пагинацию и документацию. Отчет подготовлен по структуре предоставленного примера на основе работающего проекта.

Kittygram собран и запущен локально в Docker Desktop на macOS. Gunicorn 23.0.0 обслуживает API, WhiteNoise — статику админки и ReDoc. Контейнер kittygram-web-1 работает в состоянии healthy. Проверка Django и 29 автоматических тестов завершились успешно.

В рабочей базе находятся 14 котов, 16 пользователей и 3 достижения. Владелец anna видит 6 своих котов, boris — 5, наблюдатель viewer — 1. Во время проверки создавался отдельный кот «Мурка» с ID 18; на рисунке он входит в список anna. После проверки этот кот удален.

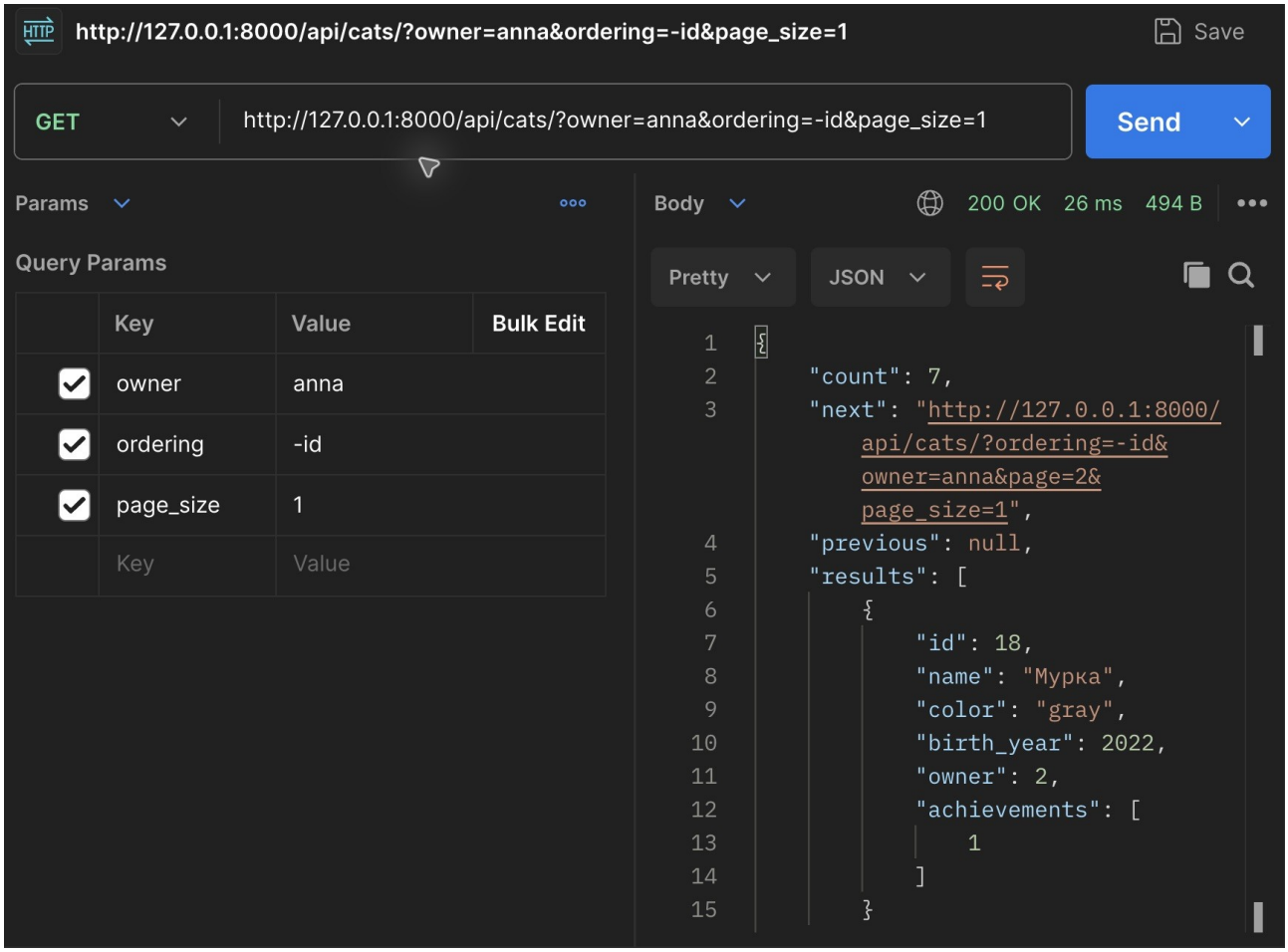


Рисунок 1 - Личный список anna с пагинацией в Postman и ответом HTTP 200

Личный API требует входа и показывает только доступные пользователю записи. Это соответствует требованию о доступе к своему коту. Функции ранее выполненной темы «Кот дня» сохранены, но в этой работе рассматривается базовый практикум.

2 Модель данных и связи

2 1 Основные модели

Модель	Поля и назначение	Связи	Ограничения
Cat	id, name, color, birth_year, owner, achievements; личная запись кота.	owner → User; achievements ↔ Achievement.	Имя 2–16; цвет 3–16; год 1900 –2026 либо null.
Achievement	id, name, description; справочник достижений.	cats ↔ Cat, связь многие ко многим.	name уникально, максимум 64 символа; description может быть пустым.
User	Стандартная модель Django: id, username, password, is_staff, is_superuser.	cats ← Cat; группы и права; Token.	Правила встроенной модели; пароль хранится в виде хеша.
Token	key, user, created; ключ для запросов API.	user → User, связь один к одному.	Один токен на пользователя; передается в заголовке Authorization.

Cat.owner хранит внешний ключ. В ответах API owner — числовой ID пользователя: у anna он равен 2. При создании владелец назначается сервером из request.user; входной owner игнорируется как поле только для чтения. В старой базе возможны записи без владельца; их видит администратор.

2 2 Бизнес правила и валидации

Правило	Где проверяется	Результат нарушения
Имя от 2 до 16 символов	CharField и MinLengthValidator; ограничения попадают в ModelSerializer.	HTTP 400, ошибка в поле name.
Цвет от 3 до 16 символов	CharField и MinLengthValidator модели Cat.	HTTP 400, ошибка в поле color.
Год от 1900 до текущего года либо null	Cat.clean и CatSerializer.validate_birth_year. В день проверки верхняя граница — 2026.	HTTP 400, ошибка birth_year.
Достижения должны существовать; название уникально	ID связи проверяет сериализатор; unique=True у Achievement.name.	Неверный ID — 400; дубликат имени блокирует модель/админка.

Миграция cats.0004 создает Achievement, добавляет связь achievements и уточняет ограничения полей. У ранее созданных котов список достижений пустой до выбора владельцем. Исторические данные не удаляются автоматически.

3 API контракт

Базовый адрес: http://127.0.0.1:8000. Для защищенных запросов используется заголовок Authorization: Token <токен>. Поддерживаются также сессия Django и Basic Authentication. Слово «владелец» ниже означает пользователя с правами группы «Владельцы котов».

Метод и путь	Назначение и права	Коды
GET /api/cats/	Список своих котов; наблюдатель читает свои записи; администратор видит все.	200, 400, 401, 403, 404
POST /api/cats/	Создание; владелец или администратор. owner назначается сервером.	201, 400, 401, 403
GET /api/cats/{id}/	Карточка своего кота; администратор читает любую.	200, 401, 403, 404
PUT /api/cats/{id}/	Полная замена; владелец своего кота или администратор. Обязательны name и color.	200, 400, 401, 403, 404
PATCH /api/cats/{id}/	Частичное изменение; владелец своего кота или администратор.	200, 400, 401, 403, 404
DELETE /api/cats/{id}/	Удаление своего кота или любого у администратора. История рейтинга защищена.	204, 400, 401, 403, 404
GET /api/achievements/	Публичный справочник достижений, только чтение.	200, 404
GET /api/achievements/{id}/	Публичная карточка достижения.	200, 404
POST /api/auth/token/	Получение токена по username и password.	200, 400
GET /api/users/me/	Собственный профиль и группы; требуется вход.	200, 401

Код 404 у списка возможен при запросе несуществующей страницы. Для владельца чужой ID также возвращает 404, так как запись исключается из queryset. Пользователь без разрешения на операцию получает 403.

Дополнительные адреса: /api/cats/mine/ — собственные коты, включая администратора; /api/docs/ — ReDoc; /api/openapi.json — схема; /health/ — проверка доступности; /admin/ — административная панель. Редактирование справочника достижений выполняет администратор через /admin/.

3 1 Примеры CRUD для кота

Ниже приведены фактические запросы от anna к контейнеру и ответы сервера. Во всех пяти запросах чтения и записи передавался ее токен. ID 18 относится к временному коту, созданному для этой проверки.

Создание кота

POST /api/cats/

Тело запроса

{"name": "Мурка", "color": "orange", "birth_year": 2022, "achievements": [1]}

Ответ 201

{"id": 18, "name": "Мурка", "color": "orange", "birth_year": 2022, "owner": 2,
"achievements": [1]}

Получение карточки

GET /api/cats/18/

Ответ 200

{"id": 18, "name": "Мурка", "color": "orange", "birth_year": 2022, "owner": 2,
"achievements": [1]}

Полная замена данных

PUT /api/cats/18/

Тело запроса

{"name": "Мурка", "color": "white", "birth_year": 2022, "achievements": [1]}

Ответ 200

{"id": 18, "name": "Мурка", "color": "white", "birth_year": 2022, "owner": 2,
"achievements": [1]}

Частичное изменение

PATCH /api/cats/18/

Тело запроса

{"color": "gray"}

Ответ 200

{"id": 18, "name": "Мурка", "color": "gray", "birth_year": 2022, "owner": 2,
"achievements": [1]}

Удаление проверочной записи

DELETE /api/cats/18/

Ответ 204

Пустое тело ответа.

Повторный GET /api/cats/18/ после удаления вернул 404: {"detail":"Страница не найдена."}

3 2 Поиск достижения и профиль

Список с поиском и фильтрами

GET /api/cats/?search=Мурка&owner=anna&birth_year=2022
&ordering=name&page_size=1

Ответ 200. Поля count, next, previous и results составляют стандартный ответ пагинации. На момент запроса тестовый кот имел цвет orange.

{"count": 1, "next": null, "previous": null, "results": [{"id": 18, "name": "Мурка",
"color": "orange", "birth_year": 2022, "owner": 2, "achievements": [1]}]}

Справочник достижений

GET /api/achievements/ → 200

{"count": 3, "next": null, "previous": null, "results": [{"id": 1,
"name": "Знает команду ко мне", "description": "Приходит по зову хозяина."}, {"id": 2,
"name": "Ловит игрушечную мышь",
"description": "Уверенно ловит и приносит игрушку."}, {"id": 3, "name": "Путешественник",
"description": "Спокойно переносит поездки в переноске."}]}

GET /api/achievements/1/ → 200

{"id": 1, "name": "Знает команду ко мне", "description": "Приходит по зову хозяина."}

Вход и собственный профиль

POST /api/auth/token/ → 200

{"username": "anna", "password": "<скрыт>"}

{"token": "<скрыт>"}

Значения пароля и токена скрыты в отчете. Полученный токен используется в Authorization. Профиль не возвращает пароль или токен.

GET /api/users/me/ → 200

{"id": 2, "username": "anna", "groups": ["Владельцы котов"], "is_staff": true,
"is_superuser": false}

4 Ключевые решения реализации DRF

CatSerializer преобразует записи Cat в JSON, проверяет поля и принимает достижения как массив существующих ID. owner доступен только для чтения. AchievementSerializer возвращает id, name и description. MyProfileView берет request.user и не позволяет запросить произвольного пользователя.

CatViewSet реализует CRUD, AchievementViewSet — чтение справочника. Они подключены через DefaultRouter. В perform_create владелец назначается из request.user. Перед поиском и сортировкой get_queryset ограничивает записи владельцем; исключение сделано только для суперпользователя.

Возможность	Реализация и параметры
Поиск	SearchFilter, поля name и color; пример search=Мурка.
Сортировка	OrderingFilter: id, name, birth_year. Префикс минус задает обратный порядок, например ordering=-id.
Фильтры	color — точное совпадение; owner — логин; birth_year — число 1900…текущий год. Ошибочный год возвращает 400.
Пагинация	ShowcasePagination: по умолчанию 5 записей; page_size до 100; параметр page выбирает страницу.
Запросы к базе	select_related для владельца и prefetch_related для достижений уменьшают число дополнительных запросов.

Фильтр owner=boris при токене anna дал count=0. При page_size=1 список anna во время проверки содержал 7 записей, включая Мурку, и ссылку next на вторую страницу. После удаления временного кота у anna снова 6 записей.

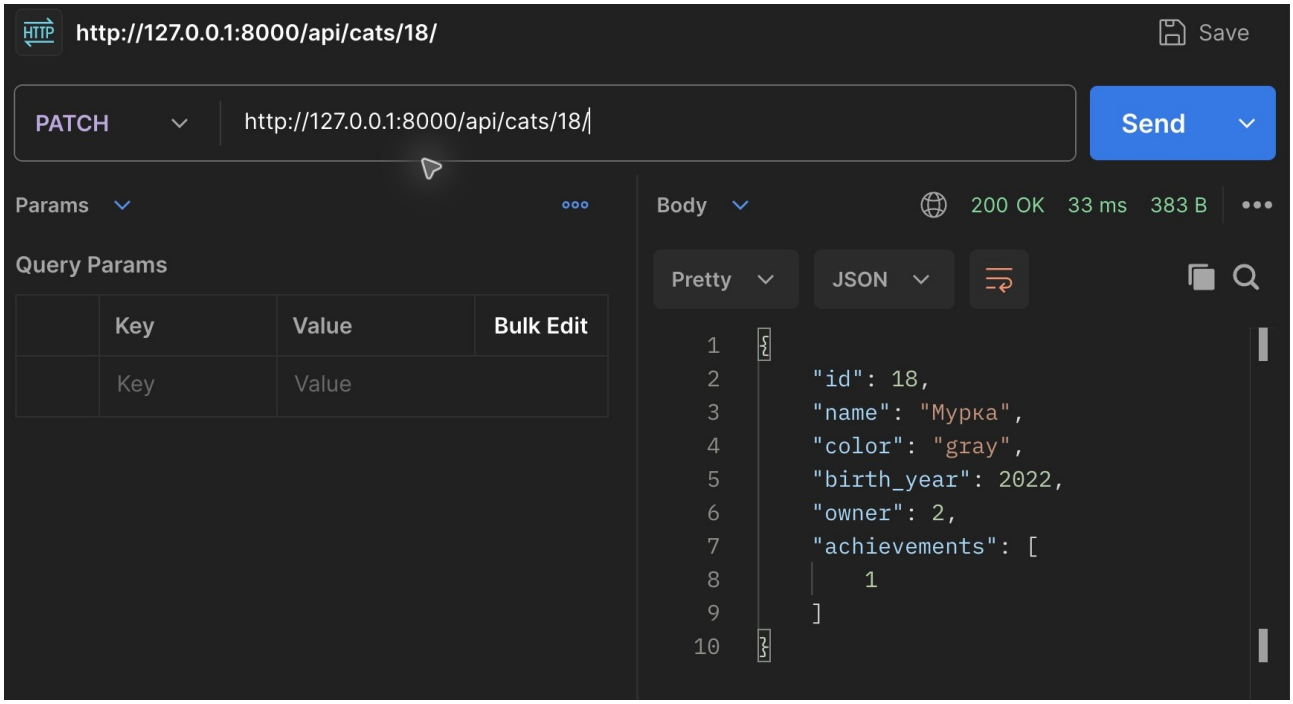


Рисунок 2 - PATCH личного кота в Postman завершился HTTP 200

5 Контроль доступа и обработка ошибок

CatModelPermissions проверяет разрешения группы для HTTP-метода, IsCatOwnerOrSuperuser — владельца объекта. Ограничение queryset скрывает чужие записи. Флаг is_staff разрешает вход в админку, но сам по себе не предоставляет доступ к чужим котам.

Роль	Чтение котов	Создание и изменение	Админка
anna / boris	Только свои	Свои записи	Свои коты
viewer	Только свой кот	Запрещено	Просмотр своего кота
demo_admin	Все записи	Все записи	Коты, достижения, пользователи, группы
Без входа	Личный API закрыт	Запрещено	Вход требуется

Сценарий	Запрос	Фактический код
Неверные имя, цвет и год	POST /api/cats/ от anna	400
Создание без токена	POST /api/cats/ анонимно	401
Создание наблюдателем	POST /api/cats/ от viewer	403
Изменение чужого кота	PATCH /api/cats/18/ от boris	404

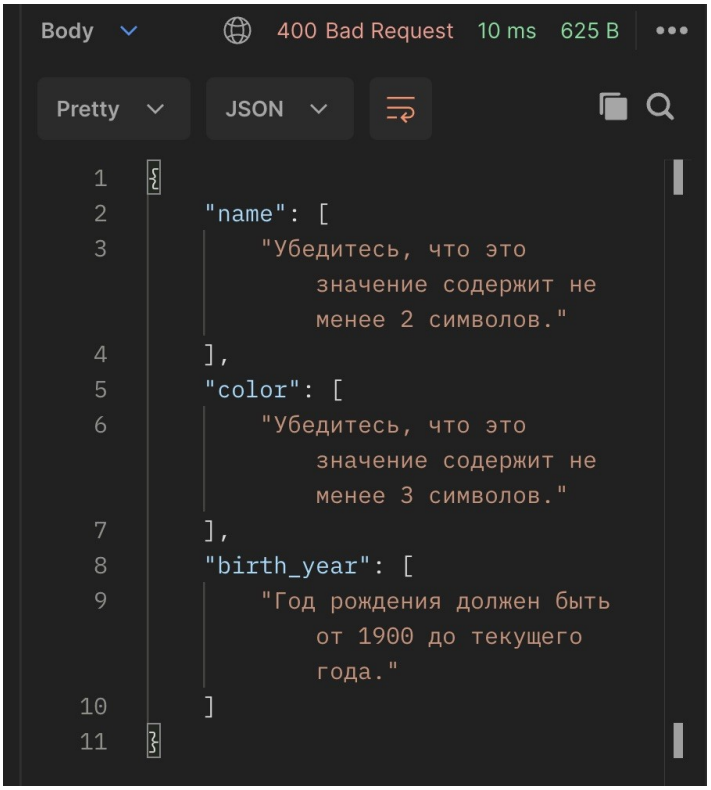


Рисунок 3 - HTTP 400 и сообщения об ошибках name color birth_year

Для чужого кота ответ: {"detail":"Страница не найдена."}. Для пользователя без токена: {"detail":"Учетные данные не были предоставлены."}. Публичные справочник достижений и витрина не открывают доступ к личной карточке.

6 Документация и проверка запросами

OpenAPI 3.0.3 генерируется drf-spectacular и доступна в JSON по /api/openapi.json. ReDoc по /api/docs/ показывает методы, поля, параметры, способы входа и коды ответов. Оба адреса вернули HTTP 200. Скрипт ReDoc хранится в проекте, поэтому страница не требует внешнего CDN.

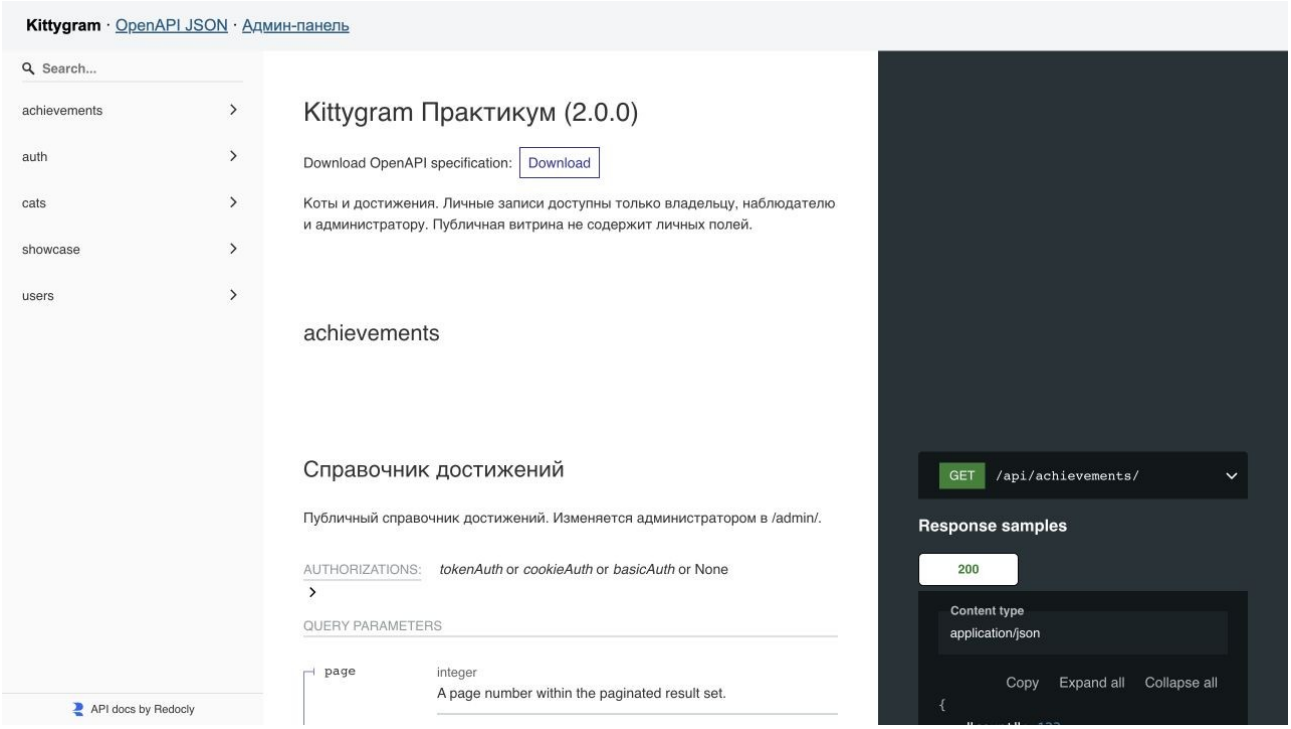


Рисунок 4 - Документация работающего Kittygram в ReDoc

Примеры в ReDoc формируются по схеме. Фактические данные работающей базы приведены в разделе 3 и файле evidence/API-responses.json. Для повторения проверки подготовлена Postman-коллекция из 22 запросов с проверками HTTP-кодов и сохранением токенов и ID во временные переменные.

Проверка	Результат
POST /api/cats/ → GET /api/cats/18/	201 при создании; 200 при чтении; owner=2, achievements=[1].
PUT и PATCH /api/cats/18/	200; полная замена color=white и частичное изменение color=gray.
DELETE /api/cats/18/ → повторный GET	204 без тела; затем 404. Исходные коты сохранены.
29 автоматических тестов	7 тестов базового практикума; 22 теста прав, админки и функций темы «Кот дня».

```
Ran 29 tests in 2.072s
OK
System check identified no issues (0 silenced).
No changes detected
```

Команда spectacular --validate --fail-on-warn завершилась с кодом 0 без предупреждений. Полные результаты сохранены в evidence/tests.txt и evidence/schema-validation.txt.

7 Деплой и Docker

Проверка выполнена в Docker Desktop 4.96.3 на macOS Apple Silicon, контекст desktop-linux. Compose собирает образ kittygram-web:local и запускает сервис web. Порт 8000 контейнера опубликован только на 127.0.0.1:8000 компьютера. HTTP-проверка /health/ дает 200, статус контейнера — healthy.

```
cd kittygram
python3 setup_env.py
docker context use desktop-linux
docker compose up -d --build --wait
docker compose ps
curl -i http://127.0.0.1:8000/health/
```

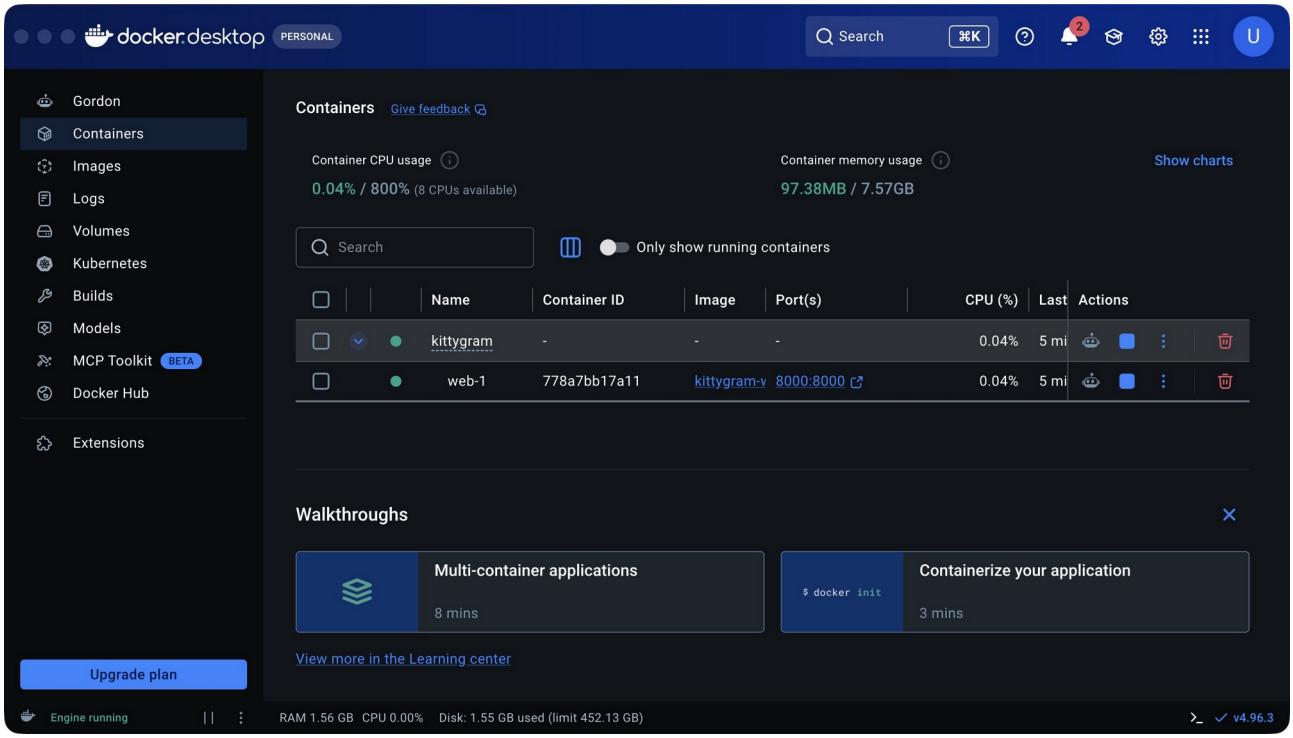


Рисунок 5 - Compose проект kittygram и работающий web в Docker Desktop

При старте entrypoint применяет миграции, настраивает группы, готовит справочник и собирает статику. Gunicorn запускается с двумя процессами. SQLite размещена в постоянном томе kittygram_data; пересоздание контейнера сохраняет данные.

```
Running migrations: No migrations to apply.
164 static files copied to '/app/staticfiles'.
Starting gunicorn 23.0.0
Listening at: http://0.0.0.0:8000
Booting worker with pid: 12
Booting worker with pid: 13
```

В showmigrations все миграции отмечены [X], включая cats.0004. Последний запуск не применял их повторно. Развертывание подтверждено для локального Docker Desktop; публикация на удаленном сервере в этой работе не выполнялась.

7 1 Воспроизводимость запуска

Исходники и пошаговый запуск описаны в README_PRACTICUM.md; команды Docker также приведены в DOCKER.md. Файл .env.example задает состав настроек. setup_env.py создает локальный .env со случайными секретами и не перезаписывает существующий файл. При первом старте с LOAD_DEMO_DATA=true создаются демонстрационные пользователи и данные.

Переменная	Назначение
DJANGO_SECRET_KEY	Случайный ключ Django, обязателен.
DJANGO_DEBUG	Учебный локальный режим отладки; по умолчанию True.
DJANGO_ALLOWED_HOSTS	Разрешенные имена: localhost, 127.0.0.1, [::1].
DEMO_PASSWORD	Начальный пароль демопользователей. Изменение переменной не меняет уже сохраненные пароли.
LOAD_DEMO_DATA	true — наполнить демонстрационными данными при первом старте.
PORT	Порт компьютера; по умолчанию 8000.

Зависимости проекта

Python 3.10; asgiref 3.3.4; Django 3.2; djangorestframework 3.12.4; pytz 2021.1; sqlparse 0.4.1; gunicorn 23.0.0; whitenoise 6.7.0; drf-spectacular 0.27.2. Версии закреплены в requirements.txt.

Команды проверки и управления

```
docker compose exec web python manage.py check
docker compose exec web python manage.py test cats showcase
docker compose exec web python manage.py makemigrations --check --dry-run
docker compose exec web python manage.py spectacular \
  --file /tmp/openapi.json --format openapi-json --validate --fail-on-warn
docker compose logs --tail=40 web
docker compose stop
docker compose up -d --wait
```

Для входа используются anna, boris, viewer и demo_admin с первоначальным паролем из локального .env. Пароль не включен в отчет. В Postman нужно заполнить переменную demo_password; первые три запроса получают токены. Запросы CRUD создают и удаляют отдельного кота без истории голосования.

Сохранение данных

После проверки в базе остались 14 исходных котов, 16 пользователей, 3 достижения, 52 дневные записи и 109 голосов. Статистика сохранена в evidence/data-summary.json. Файлы .env, база SQLite и реальные токены исключены из архива материалов.

8 Итоги и самооценка

Выполнен базовый практикум Kittygram: реализованы достижения и связь с котами, профиль текущего пользователя, токеновая аутентификация, CRUD, проверки полей и прав доступа. Поиск, сортировка и пагинация работают с ограниченным по владельцу набором записей. Администратор управляет всеми данными, наблюдатель читает только своего кота.

Проверены ответы 200, 201, 204, 400, 401, 403 и 404. Все 29 автоматических тестов прошли. Проект запускается в Docker Desktop с Gunicorn; документация ReDoc доступна локально. Следующее улучшение — подготовка отдельной конфигурации для публикации на удаленном сервере с HTTPS и внешней СУБД.

Соответствие критериям текущего рейтинга

Критерий	Макс	Подтверждение в работе
Оформление	1	Восемь разделов по примеру, титульный лист, пять скриншотов, ссылки и JSON.
Запуск	3	README_PRACTICUM.md, .env.example, setup_env.py и проверенный запуск Docker.
Модель данных	2	Cat, Achievement, User, Token; связи и четыре группы ограничений.
API контракт	4	10 основных операций с правами, кодами и фактическими JSON.
DRF реализация	4	Сериализаторы, viewsets, router, permissions, фильтры и пагинация.
Документация и запросы	3	OpenAPI, ReDoc, 22 запроса Postman, полная CRUD-проверка и 29 тестов.
Деплой и Docker	3	Локальный Docker Desktop, Gunicorn, healthcheck, миграции и журнал запуска.
Итого	20	Представлены материалы для проверки; итоговые баллы выставляет преподаватель.

Материалы для проверки

В комплект входят отчет в DOCX и PDF, исходники Kittygram, Postman-коллекция, схема OpenAPI, скриншоты и папка evidence с фактическими ответами API, результатами тестов, списком миграций и журналом Docker. Перед запуском архива следует создать собственный .env командой python3 setup_env.py.

Ссылки

Исходный Kittygram — github.com/yandex-praktikum/kittygram

Локальная документация — http://127.0.0.1:8000/api/docs/

Административная панель — http://127.0.0.1:8000/admin/

Django REST Framework — документация по схемам

drf spectacular — официальная документация